Module 4: Object-Oriented and Concurrent Languages

Topics

1. Object-Based Language Features
2. Object-Oriented Language Features
3. Concurrent Language Features
4. Exceptions

I. Object-Based Language Features

The term *object-based language* is sometimes used to refer to languages that provide the syntax for some, but not all, of the features required of true object-oriented languages. Ada 83 falls into this category.

A. Abstraction, Encapsulation, and Information Hiding

As programs grew larger throughout the history of software development, various levels of abstraction developed as a way to manage ever-larger programs. Process abstraction has been incorporated in high-level languages from the very beginning. Data abstraction is a more recent development.

For each level of abstraction in any language that supports it, there is an associated syntactic mechanism of encapsulation and another kind of support for information hiding. Some texts use *encapsulatio*n and *information hiding* as synonyms. Others make what we consider an important distinction between the two, which will be our approach.

- **Encapsulation** involves grouping together a collection of more elementary components into a larger whole. The language needs some syntax to define the boundaries of this grouping.

- **Information hiding** involves restricting the visibility of data. The scope rules of the language generally provide this capability.

The subprogram is the level 1 encapsulation syntax. It supports grouping together data declarations with a collection of statements that are needed to accomplish a particular task. Level 1 encapsulation was the prime focus of structured design.

At the subprogram level, all local declarations are implicitly hidden from other subprograms. In languages that do not support nested subprograms, they are hidden from all other subprograms. When nested subprograms are permitted, the rules are more complicated, as we discussed in the previous modules.

When programming languages adopted syntax to support data abstraction, level 2 encapsulation was introduced. The syntax for level 2 encapsulation in C++, Java, and C# is the *class*. A class groups together data declarations and the methods that operate on them.

At the class level, hiding information is a bit more complicated. To limit the access to instance and class data to only the methods within the class, most languages require using the reserved word private. Such strict hiding of information is sometimes too limiting when inheritance is involved. Consequently, most languages provide some technique for allowing derived classes greater access. We will elaborate on inheritance later in this module.

Table 4.1 summarizes these two levels of abstraction and the associated language features that support them.

Table 4.1
Levels of Abstraction

Level	Abstraction	Encapsulation	Information Hiding

1	process abstraction	subprogram	local data
2	data abstraction	class	private data

Integer Set in C++

To help you better understand data abstraction and the way language features support it, we will examine an example in two languages, beginning with C++. The example we have chosen is to implement a new data type for integer sets. In module 3, we presented a similar example using complex numbers to illustrate operator overloading. Now, we concentrate on aspects related to data abstraction.

One language feature related to data abstraction that varies among languages is whether the specification is separated from the implementation. C++ and Ada make such a separation. As we saw in module 3, it is customary to place the class definition in a header file—a file whose name is the class name with the file extension .h. The bodies of the member function, by convention, are placed in a file whose name is the class name with the file extension .cpp. In all languages, separate source files serve as a kind of an encapsulation mechanism, although their use is dictated by convention, not as a part of the language definition.

Let's consider the class definition for the C++ implementation of an integer set data type whose values can be the integers from 1 to 25.

Move your mouse over the highlighted parts of the class definition to see the explanations of those parts.

```
const int SIZE = 25;

class Int25Set
{
public:
    Int25Set();
    void add(int value);
    bool in(int value) const;
    Int25Set operator&(Int25Set right)  const;
private:
    bool inRange(int position) const
    {
        return (position >= 0 && position < SIZE);
    }
    bool set[SIZE];
};
```

Next, consider the bodies of the functions, which by convention would be placed in a file called int25set.cpp.

Move your mouse over the highlighted parts of the function bodies to see the explanations of those parts.

```
#include "int25set.h"

Int25Set::Int25Set()];
{
    for (int i = 0; i < SIZE; i++)
        set[i] = false;
}

void Int25Set::add(int value)
{
    int position = value - 1;
    if (inRange(position))
        set[position] = true;
}

bool Int25Set::in(int value) const
{
    int position = value - 1;
    return inRange(position) && set[position];
}

Int25Set Int25Set::operator&(Int25Set other) const
{
    Int25Set result;

    for (int i = 0; i < SIZE; i++)
        result.set[i] = set[i] & other.set[i];
    return result;
}
```

Notice that there is really no syntactic encapsulator for the function bodies, which is why using separate files is so important.

Finally, let's examine a simple main function that will use several of the functions. The program creates two sets and then adds a 1 to the first set and a 2 to the second. It then forms their intersection, assigns it to a third set, and confirms that the third set contains neither a 1 nor a 2.

```
#include <iostream>

using namespace std;

#include "int25set.h"

int main()
{
    Int25Set set1, set2, set3;

    set1.add(1);
    set2.add(2);
    set3 = set1 & set2;
    if (!set3.in(1) && !set3.in(2))
        cout << "set3 is empty as it should be." << endl;
    return 0;
}
```

Differences in Java and C#

Although we do not intend to provide the code for this example in Java and C#, we will explain the ways in which that code would differ. The first difference is that neither Java nor C# separates the class definition from the method bodies as C++ does. In both Java and C#, the method bodies are defined within the class definition.

The next difference is with regard to operator overloading. Java does not allow it, so a named method, for example, intersection in place of operator&, would be used instead. Although C# allows operator overloading, it does not allow friend

functions, so a static method must be used instead. Using a static method would require two parameters, just like using a friend function in C++ does.

Integer Set in Ada

Let's now consider the same integer data set implemented in Ada. We begin with the specification. In Ada, specifications are put in a package specification. Unlike classes, packages may define more than a single type although, customarily, packages are used to define a single type. Including other types is a good practice only if they are tightly related to the primary type.

Move your mouse over the highlighted parts of the package specification to see the explanations of those parts.

```
package Int25_Set_Package is
    type Int25 is range 1..25;
    type Int25_Set is private;
    procedure Add (Set: in out Int25_Set; Value: in Int25);
    function In_Set(Set: in Int25_Set; Value: in Int25) return
    function "and"(Left, Right: Int25_Set) return Int25_Set;
private
    type Int25_Array is array(Int25) of Boolean;
    type Int25_Set is
        record
            Set_Array: Int25_Array := (others => false);
        end record;
end Int25_Set_Package;
```

It is customary, although not required, to separately compile package specifications and bodies, placing each in its own source file. When separate compilation is used, the package specification must be compiled before its body and any other compilation unit that uses it.

Next, we examine the package body.

Move your mouse over the highlighted parts of the package body to see the explanations of those parts.

```
package body Int25_Set_Package is
    procedure Add(Set: in out Int25_Set; Value: in Int25) is
    begin
        Set.Set_Array(Value) := true;
    end Add;
    function In_Set(Set: in Int25_Set; Value: in Int25)
        return Boolean is
    begin
        return Set.Set_Array(Value);
    end In_Set;
    function "and"(Left, Right: Int25_Set) return Int25_Set is
        Result: Int25_Set;
    begin
        Result.Set_Array := Left.Set_Array and Right.Set_Array;
        return Result;
    end "and";
end Int25_Set_Package;
```

Finally, for completeness, we include a main procedure that uses this package and the type that it defines.

```
with Text_IO; use Text_IO;
with Int25_Set_Package; use Int25_Set_Package;

procedure Int25_Main is
    Set1, Set2, Set3 : Int25_Set;
begin
    Add(Set1, 1);
    Add(Set2, 2);
    Set3 := Set1 and Set2;
    if not In_Set(Set3, 1) and not In_Set(Set3, 2) then
        Put("Set3 is empty as it should be.");
        New_Line;
    end if;
end Int25_Main;
```

B. Type Parameterization

Clearly there are several limiting factors of the integer set implementation that we just considered. It can be used only for sets of integers and moreover only for integer values between 1 and 25. Of course, it is highly desirable to write code that can be reused in a wide range of situations, enabling it to be supplied by the compiler, saving programmers from having to write essentially the same code again and again.

Ideally, we would like a set type that could hold sets of any kind of elements. If we use the representation we used in the previous examples, we are somewhat limited in how much we can generalize it. Although we could certainly allow the size to be supplied to the constructor in the C++ case and have integer sets of any range, in Ada we could use a generic package and generalize the elements to be any discrete type, which is Ada's restriction on array subscripts.

To increase the reusability of this type and illustrate the way type parameterization is implemented in different languages, let's again consider developing a set type but use a somewhat different representation—a sorted list. If we choose a sorted list,

we are still limited to the type of the components, but less so compared to the previous representation. Now, the only requirement is that the set components be an ordered type so that they can be sorted.
Because type parameterization is accomplished so differently in C++, Ada, and Java, we elect to focus on examples rather than discuss the generalities of parameterized types.

Sorted Set in C++

The syntax used in C++ to parameterize types is *templates*. Because template instantiation is done at compile time and template functions cannot be fully compiled until they are instantiated, it is customary to place both the class definition and the function bodies in the header file.

Shown below is the content of the header file for a C++ implementation of a sorted set that has a template type parameter for the component type and a template constant parameter for the size. In this implementation, we have omitted the set intersection operation to keep the size of this example from becoming too large.

Move your mouse over the highlighted parts of the template class to see the explanations of those parts.

```
template<typename Component, int SIZE>
class SortedSet
{
public:
    SortedSet()
    {
        length = 0;
    }
    void add(Component item);
    bool in(Component item) const;
private:
    Component set[SIZE];
    int length;
};

template<typename Component, int SIZE>
void SortedSet<Component, SIZE>::add(Component item)
{
    if (length < SIZE)
    {
        int i = length - 1;
        while (i >= 0 && set[i] > item)
        {
            set[i + 1] = set[i];
            i--;
        }
        set[i + 1] = item;
        length++;
    }
}

template<typename Component, int SIZE>
bool SortedSet<Component, SIZE>::in(Component item) const
```

```
{
      int low = 0, high = length - 1, middle;
      while (low <= high)
      {
         middle = (low + high) / 2;
         if (set[middle] == item)
            return true;
         else if (set[middle] > item)
            high = middle - 1;
         else
            low = middle + 1;
      }
      return false;
}
```

To illustrate instantiation of a template class, shown below is a simple main function that uses the sorted set. It adds a 1 to the set and then confirms that the set contains a 1.

Move your mouse over the highlighted part of the function to see an explanation of that part.

```
#include <iostream>

using namespace std;

#include "sortedset.h"

int main()
{
    SortedSet<int, 100> set;

    set.add(1);
    if (set.in(1))
        cout << "set1 contains 1 as expected." << endl;
    return 0;
}
```

What is a particular shortcoming in C++ is that there is no way to specify any limitations on what types can be used for the template type parameters. In the above case, if we attempt to instantiate the template class using a type that does not have a > operator, the template class will not compile.

Sorted Set in Ada

Next, let's examine the same implementation of a sorted set, this time using Ada. Type parameterization is accomplished in Ada by using generic packages. Like ordinary packages, the specification and body of generic packages are usually compiled separately. First, let's examine the specification.

Move your mouse over the highlighted parts of the package specification to see the explanations of those parts.

```
generic
    type Component is private;
    with function ">"(Left, Right: Component) return Boolean is
    Size: in Integer;
package Sorted_Set_Package is
    type Sorted_Set is private;
    procedure Add(Set: in out Sorted_Set; Item: in Component);
    function In_Set(Set: in Sorted_Set; Item: in Component)
        return Boolean;
private
    type Sorted_Array is array(1..Size) of Component;
    type Sorted_Set is
        record
            Set_Array: Sorted_Array;
            Length: Natural := 0;
        end record;
end Sorted_Set_Package;
```

Next, we examine the body of that package.

Move your mouse over the highlighted part of the package body to see the explanations of that part.

```
package body Sorted_Set_Package is
    procedure Add(Set: in out Sorted_Set; Item: in Component) i
        I: Integer;
    begin
        if Set.Length <= Size then
            I := Set.Length;
            while I >= 1 and Set.Set_Array(I) > Item loop
                Set.Set_Array(I + 1) := Set.Set_Array(I);
                I := I - 1;
            end loop;
            Set.Set_Array(I + 1) := Item;
            Set.Length := Set.Length + 1;
        end if;
    end Add;
    function In_Set(Set: in Sorted_Set; Item: in Component)
        return Boolean is
        Low: Integer := 1;
        High: Integer := Set.Length;
        Middle: Integer;
    begin
        while Low <= High loop
           Middle := (Low + High) / 2;
           if Set.Set_Array(Middle) = Item then
              return true;
           elsif Set.Set_Array(Middle) > Item then
              High := Middle - 1;
           else
              Low := Middle + 1;
           end if;
        end loop;
      return false;
```

```
    end In_Set;
end Sorted_Set_Package;
```

To conclude this Ada example, consider a simple main procedure that instantiates the generic package.

Move your mouse over the highlighted part of the procedure to see an explanation of that part.

```
with Text_Io; use Text_IO;
with Sorted_Set_Package;

procedure Sorted_Set_Main is
    package Sorted_Int_Set_Package is new
      Sorted_Set_Package(Component => Integer,  Size => 100);
    use Sorted_Int_Set_Package;
    Set: Sorted_Set;
begin
    Add(Set, 1);
    if In_Set(Set, 1) then
       Put ("Set contains 1 as expected.");
       New_Line;
    end if;
end Sorted_Set_Main;
```

The compilation order rules are stricter with generic packages. Both the specification and the body must be compiled before any compilation unit that contains an instantiation of the generic package.

Sorted Set in Java

We conclude our discussion of type parameterization by examining how the sorted set would be implemented in Java. Before version 5.0 of Java, the approach to writing type-independent container classes was to make the component be type Object when possible, or, if necessary, to use a more restrictive interface type. This

approach is possible in Java because all objects are heap-dynamic so the compiler need not concern itself with the actual size of the objects. That issue is especially important in both C++ and Ada, which support stack-dynamic data, because code with type parameterization can never be fully compiled until it is instantiated in those languages.
The one drawback to Java's initial approach was that when objects were removed from a container, to be usable they had to be downcast back to their original type. Downcasts are potentially unsafe and can be checked only at run time. The syntax for type parameterization added in version 5.0 eliminates the need to downcast and enables type checking at compile time.

Shown below is a Java version of the sorted set. Rather than including a method to search for an element in the set, we included a method to extract the element at a specific index so we could illustrate the place where downcasting is no longer needed in the client class.

Move your mouse over the highlighted parts of the class to see the explanations of those parts.

```
class SortedSet <Component extends Comparable>
{
    private  Component[] set;
    private int length, size;

    public SortedSet(int size)
    {
        set = (Component[]) new Comparable[size];
        this.size = size;
        length = 0;
    }
    public void add(Component item)
    {
        if (length < size)
        {
            int i = length - 1;
            while (i >= 0 && set[i].compareTo(item) > 0)
            {
                set[i + 1] = set[i];
                i--;
            }
            set[i + 1] = item;
            length++;
       }
    }
    public Component get(int index)
    {
        if (index < 0 || index >= length)
            return null;
        return set[index];
    }
}
```

To conclude this example, we supply a simple class containing a main method that instantiates a sorted set object.

Move your mouse over the highlighted parts of the class for an explanation.

```
class SortedSetMain
{
    public static void main(String[] args)
    {
        SortedSet<Integer> set = new SortedSet<Integer>(100);

        set.add(1);
        Integer first = set.get(0);
        System.out.println("The first element in the set is " +
    }
}
```

II. Object-Oriented Language Features

For a language to be considered a fully object-oriented language, in addition to providing support for encapsulation and information hiding, it must support inheritance and run-time polymorphism. It is those latter two features that we consider next.

In this section, we have elected to confine our examples to C++ only. We intend to highlight the ways in which a similar program written in Java would differ. We will also mention some differences in C#. Ada 83 does not support inheritance, but it was added in Ada 95. As is often the case when features are subsequently added to a language, the addition is less than seamless. For that reason, we plan to avoid any discussion of Ada 95 in this section.

A. Inheritance

In module 2, we discussed variant records and provided an example of an Ada variant record—a person that had two variants: a single person and a married person. The data relationships implemented with variant records are the same relationships for which inheritance is best suited—relationships characterized by the *is a* predicate. In this case, we have:

- A single person *is a* person.
- A married person *is a* person.

Let's consider the way this example would be implemented in C++ with inheritance, considering only the married variant. In the previous section, as is customary, we separated our programs into multiple files. Recall, however, that doing so is not a requirement. For the sake of brevity and to avoid all the include statements, our examples in this section will be presented as single file programs.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
#include <string>
using namespace std;

class Person
{
public:
    Person(string name, int age);
    void put(ostream& out);
private:
    string name;
    int age;
};

Person::Person(string name, int age)
{
    this->name = name;
    this->age = age;
}
void Person::put(ostream& out)
{
    out << "name = " << name << " age = " << age;
}

class Married: public Person
{
public:
    Married(string name, int age, string spouse);
    void put(ostream& out);
private:
    string spouse;
};
```

```
Married::Married(string name, int age, string spouse)
    : Person(name, age)
{
    this->spouse = spouse;
}

void Married::put(ostream& out)
{
    Person::put(out);
    out << " spouse = " << spouse;
}

int main()
{
    Married ted("Ted", 25, "Sally");
    Person* person = &ted;

    ted.put(cout);
    cout << endl;
    person->put(cout);
    cout << endl;
    return 0;
}
```

Finally let's consider the output of this program, which is shown below:

```
name = Ted age = 25 spouse = Sally
name = Ted age = 25
```

If you are most familiar with Java, this output may surprise you. When the put method is invoked on the pointer person, the put method of the Person class is called as is evident from the output. The type of the pointer is used to determine which function is called, not the type of the object. Consequently, this determination is made at compile time.

B. Polymorphism

In C++, when inheritance is used, run-time polymorphism is not the default as our previous example illustrates. To achieve run-time polymorphism, we must modify very slightly the base class definition of the previous program. We have highlighted the one change, which is the addition of the reserved word virtual to the prototype of the put function. That modified version is shown below:

```
class Person
{
public:
    Person(string name, int age);
    void virtual put(ostream& out);
private:
    string name;
    int age;
};
```

With this simple modification, the output of the program now becomes the following:

```
name = Ted age = 25 spouse = Sally
name = Ted age = 25 spouse = Sally
```

The invocation of the put function on the person pointer now exhibits run-time polymorphic behavior. The type of the object determines which put function is called, not the type of the pointer. When a virtual function is redefined in a derived class, it is called *overriding*.

In C#, the reserved word override must be used when we wish to override virtual methods. Java does not make a distinction between virtual and nonvirtual methods. In Java, all overridden methods exhibit the behavior of virtual methods. No designation using virtual is required, however.

C. Inheritance and Information Hiding

Including inheritance in a language interacts with the desire to keep the representation of a data type hidden. The member functions of derived classes must often access the data members of their base class. In our previous example, the put function in Married cannot access the data member name in the base class Person. To allow such access, declaring it in a region labeled protected would be necessary.

Java uses the same reserved word, but in Java it has a slightly different meaning. In Java, labeling data or methods as protected makes them accessible to any method in a class in the same package and any method in any derived class, regardless of what package it is in. Java has one more access mode, in addition to public, protected, and private. By omitting any of the three modifiers, we give access to any method in any class in the same package.

D. Inheriting Only the Specification

Although when inheritance was initially introduced there was disagreement on how it was best used, today most software designers agree that it should most often be used for *is a* relationships. In those cases, we wish to inherit both the specification and the representation.

There is one other time when inheritance is widely considered appropriate, which is when we wish to inherit only the specification. This kind of inheritance is so important that Java distinguishes it in two ways. The reserved word interface is used to define classes that contain only a specification, and the reserved word implements is the predicate that connects the interface with the class implementing it. In a Java interface, all the methods are public and abstract, although neither modifier is required. Any data contained in an interface must be public, static, and final.

We believe the differences between interfaces and classes are important enough that the use of these other two reserved words, interface and implements, becomes a wise language design decision. Being an older language, C++ makes no such distinction. That does not mean, however, that interfaces and interface inheritance cannot be done in C++. The following example illustrates that it can.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
template <typename Component>
class Stack
{
public:
    virtual void push(Component value) = 0;
    virtual Component pop() = 0;
};

const int SIZE = 100;

template <typename Component>
class ArrayStack : public Stack<Component>
{
public:
    ArrayStack();
    void push(Component value);
    Component pop();
private:
    Component stack[SIZE];
    int top;
};
```

An abstract class lies between an ordinary class and an interface. In Java, such classes are labeled with the reserved word abstract. These classes are permitted to have abstract methods, which then must also be labeled with the reserved word abstract. Unlike interfaces, they can have ordinary methods and variables.

C++ does not require a formal designation of classes that are abstract. The presence of a pure virtual function is all that is required to designate a class as abstract.

In both languages, every abstract method must be defined eventually along every branch of the inheritance hierarchy. Any derived class that does not define all the abstract methods it inherits is itself an abstract class. Although abstract classes can have constructors, only the constructors of the derived classes can call them. No objects of abstract classes can be instantiated.

E. Inheriting Only the Representation

One final use of inheritance is when inheriting only the representation is desired. Not everyone agrees that such uses of inheritance are appropriate. One thing is certain, however; when inheriting only the representation is desired, one should use inheritance only if a language provides the necessary support. C++ provides such support. Java does not.

To illustrate this use of inheritance in C++, we show the way a stack class could be written that is derived from the predefined class vector in the C++ standard template library. A program illustrating its use is presented below.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <vector>
using namespace std;

template<typename Component>
class Stack: private vector<Component>
{
public:
    void push(Component item)
    {
        add_back(item);
    }
    Component pop()
    {
        return remove_back();
    }
};

int main()
{
    Stack<int> stack;

    stack.push(1);
    stack.add_back(2);
    return 0;
}
```

Although Java does not provide support for properly inheriting only the representation, the Java Stack class is implemented in just the way we implemented our Stack class in C++ above. As a result, one can write programs like the following one in Java.

Move your mouse over the highlighted part of the program to see the explanation of that part.

```
import java.util.*;

class JavaStackDesign
{
    public static void main(String[] args)
    {
        Stack<Integer> stack = new Stack<Integer>();

        for (int i = 0; i < 10; i++)
            stack.push(i);
        System.out.println(stack.get(4));
    }
}
```

F. Multiple Inheritance

One final design decision that any object-oriented language must make is whether to support multiple inheritance. C++ allows multiple inheritance of classes, but Java does not. Java does, however, allow multiple inheritance of interfaces.

To understand Java's design decision, it is helpful to understand some of the complexity that arises when multiple inheritance of classes is permitted. One complication is when two or more base classes have functions with identical prototypes. In that case, calls to that function become ambiguous. C++ compilers detect such ambiguity and generate compilation errors. Prepending the name with the name of the desired base class resolves the ambiguity.

Another complication occurs when two or more base classes have a common parent. The question becomes whether only one copy of the data members of the common parent should be inherited or whether multiple copies should be inherited. In C++, the presence or absence, respectively, of the reserved word virtual in the inheritance clause allows the programmer to specify this choice. We should note

that the use of virtual in this context is the reuse of a reserved word in a way that has little relationship to how it is used in the previously mentioned context—a language design decision that we have often derided.
These complications and others are eliminated when multiple inheritance is limited to interfaces because interfaces can contain only method signatures and constant data.

III. Concurrent Language Features

Adding support for concurrency in languages is a more recent trend, so older languages such as C and C++ have no such support. Even if you have studied Java, which supports concurrency, it is generally considered an advanced concept so it may be new to you. Nonetheless, it is a concept that you should not find difficult. The primary difference between a sequential program and a concurrent program is that a sequential program has a single thread of control and a concurrent program has several threads of control that run at the same time—or at least appear to.

Among the languages that have support for concurrency, Ada has the richest syntax. For that reason, we have elected to confine our discussion in this section exclusively to Ada examples. In the case study for this module, we have chosen a Java program that involves concurrent execution.

A. Underlying Implementation of Concurrency

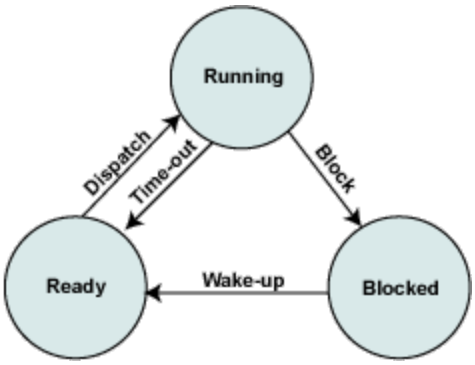
Keep in mind that programs involving concurrency can be written in languages that provide no syntactic support for concurrency by making operating-system calls to implement the required concurrent behavior. When a language does provide syntactic support for concurrency, the programs have the advantage of being operating-system independent.

Although it is not absolutely necessary to understand the underlying operating system implementation in the latter case, it is still somewhat instructive, so we begin with a brief discussion of that topic. There are two fundamental techniques for implementing concurrent execution—physical concurrency and logical concurrency.

Physical concurrency is the simpler of the two implementations. In such a system, a separate processor executes each task. Although the configuration may vary in such systems, each processor generally has some separate memory, and all processors share other memory.

With logical concurrency, a multitasking operating system appears to be simultaneously executing multiple tasks. Let's briefly discuss how a multitasking operating system accomplishes this illusion. To understand the operation of a multitasking operating system, you must understand the role of its scheduler. The scheduler decides which task should run next. To understand the way it decides what should run, you must understand the various states of a task and the events, which cause state transitions to occur. Figure 4.1 illustrates the task states and transitions.

Figure 4.1
Task State Transition Diagram



A task is in running state when it is using the processor. It is in the ready state if it is ready to use the processor but not currently using it. The third state, blocked, requires more explanation. A task is blocked when it is waiting for some event to occur. The most common event a task awaits is I/O completion. Because I/O is a slow process, a multitasking operating system will suspend a task when it makes an I/O request until the request has been completed. Doing so enables another task to use the processor during that interval. The illusion of concurrent execution is achieved by allowing another task to run while a first task is awaiting an event.

There are other events for which tasks can wait. Among them are messages from other tasks, expiration of a timer, or an interrupt from an external device. When a task reaches a point where it must await some event, the task relinquishes the

processor, leaves the ready state, and enters the blocked state.

B. Reasons for Synchronization

The need for synchronization is one of the key differences between programs that involve concurrency and those that do not. Concurrent programs need synchronization for the following two fundamental reasons:

1. to provide the cooperation between tasks that is sometimes needed
2. to avoid an undesirable kind of competition between tasks

Cooperation Synchronization

We begin by discussing cooperation synchronization. Whenever one task must wait until another task has performed some activity before it can proceed, cooperation synchronization is needed. The example we have chosen to illustrate this kind of synchronization will also serve as a means to compare concurrent and sequential programs and to introduce the syntax for Ada tasks.

The example involves the activities one must perform upon arrival at an airport. On arrival at the airport, one must perform two activities. We are intentionally using *activity* in place of *task* to avoid confusion with Ada tasks. The two activities are:

1. pick up luggage
2. obtain a rental car

If we are traveling alone, we must perform these activities in sequence. The order of these activities is unimportant, so we may choose any sequence. First, we implement this problem with a sequential program. It is convenient to assume the existence of a package that implements the details. For obvious reasons, we choose to omit the details of that package. Shown below is the code for performing the two activities.

```
with Airport_Package; use Airport_Package;
procedure Airport_Arrival_1 is
begin
    Pickup_Luggage;
    Get_Rental_Car;
end Airport_Arrival_1;
```

If we are traveling with a friend, the activities can be performed in parallel. One person can perform one of the activities while at the same time the other person performs the other activity. Each task executes a separate subprogram. The fact that the order of these activities is irrelevant is critical to their being able to be done in parallel. In this example, we have a main procedure that contains two subordinate tasks as shown below.

```
with Airport_Package; use Airport_Package;
procedure Airport_Arrival_2 is
     task Luggage_Task;
     task Rental_Car_Task;
     task body Luggage_Task is
     begin
         Pickup_Luggage;
     end Luggage_Task;
      task body Rental_Car_Task is
     begin
         Get_Rental_Car;
     end Rental_Car_Task;
begin
    null;
end Airport_Arrival_2;
```

One important characteristic of Ada tasks that is important to recognize is that they do not need to be explicitly started. In the above program, merely declaring them inside a procedure causes them to begin executing simultaneously when the

composite procedure begins execution. Furthermore, the order in which they are declared is not significant.

Finally, this example illustrates the most rudimentary kind of cooperative synchronization. It guarantees that the Airport_Arrival_2 procedure will not complete until both of its component tasks have completed. Ada has a rich syntax for more sophisticated cooperative synchronization. We will explore some of that syntax shortly.

Competition Synchronization

Competition synchronization is needed when two or more tasks must access common data. Failure to perform proper synchronization can lead to programs that may execute differently depending on the timing of events. Such programs are said to contain a *race condition*.

As an example of a race condition, let's consider a simple two-task program in which both tasks access a shared variable. The specific example that we choose to illustrate this problem is a pair of automated teller machines. The customer's checking account will be implemented as a package with two procedures: Deposit and Withdraw. Both subprograms access the checking-account balance, which is the shared variable. Finally, there are two identical tasks that implement the two ATM machines. Both of these tasks can access the checking-account package. Let us consider the main procedure that contains these tasks.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
procedure Race_Condition is
    package Account is
        procedure Deposit(Amount: in Integer);
        procedure Withdraw(Amount: in Integer; Sufficient_Funds
            Boolean);
    private
        Balance: Integer := 0;
    end Account;
    package body Account is separate;
    task type ATM;
    task body ATM is separate;
    ATM_1, ATM_2: ATM;
begin
    null;
end Race_Condition;
```

Shown below is the body of the Account package, which should require no explanation.

```
separate(Race_Condition)
package body Account is
    procedure Deposit(Amount: in Integer) is
    begin
        Balance := Balance + Amount;
    end Deposit;
    procedure Withdraw(Amount: in Integer; Sufficient_Funds: ou
            Boolean) is
    begin
        if Amount <= Balance then
            Balance := Balance - Amount;
            Sufficient_Funds := True;
        end if;
        Sufficient_Funds := False;
    end Withdraw;
end Account;
```

To complete the programs, consider the body of the ATM task type:

```
with Text_IO; use Text_IO;
separate(Race_Condition)
task body ATM is
    type Transactions is (Deposit_Transaction, Withdraw_Transac
    package Transactions_IO is new Enumeration_IO(Transactions)
    use Transactions_IO;
    package Int_IO is new Integer_IO(Integer); use Int_IO;
    Sufficient_Funds: Boolean;
    Transaction: Transactions;
    Amount: Integer;
begin
    loop
        Put("Enter Transaction Type and Amount: ");
        Get(Transaction);
        Get(Amount);
        case Transaction is
            when Deposit_Transaction =>
                Account.Deposit(Amount);
            when Withdraw_Transaction =>
                Account.Withdraw(Amount, Sufficient_Funds);
                if not Sufficient_Funds then
                    Put("Insufficient Funds"); New_Line;
                end if;
        end case;
    end loop;
end ATM;
```

Remember that there are two instances of this task. It is best to envision each one using a separate console for its input/output.

Next, we outline the various steps leading to a race condition. To fully understand this sequence of events, we must realize that one line of Ada code is actually translated into many lines of machine code. Let us consider the assignment

statement, which is the body of the procedure Withdraw in the package Account. On most machines, this single line would translate into several lines of machine code. Figure 4.2 describes the steps that lead to a race condition.

Figure 4.2

A Scenario That Causes a Race Condition

0:00 / 1:11

The result of the sequence of events in the above figure is that $25 has been withdrawn from two ATM machines, and the balance has changed from $100 to $75 instead of the correct sum, $50.

No bank would be in business long if it allowed such race conditions to occur. Race conditions are a particularly insidious problem for concurrent programs because incorrect results are not likely to occur frequently in programs that contain race conditions. For that reason, testing is not a reliable method for discovering such problems. It is important, then, to be certain that such problems are avoided at the design stage. Although we have described a race condition in the context of a single processor system, the same problem can occur with a multiprocessor system, which contains shared memory.

The possibility of a race condition arises when a task is interrupted during a critical section. In our ATM example, the assignment statement that updates the balance is the critical section. An approach that would prohibit more than one task from having access to a given variable at the same time is an implementation of mutual exclusion. In our ATM example, if the withdraw and deposit operations have mutually exclusive access to the account balance, a race condition will be prevented.

C. Synchronization Techniques

Languages can provide several techniques for synchronization, including the use of monitors, message passing, and semaphores. Although any one of them is sufficient, languages often provide more than one. We will discuss these techniques in the following sections.

Monitors

Monitors provide synchronization that aims primarily to solve competition synchronization problems. Hoare and Brinch Hansen developed them in the mid-1970s. A monitor is a language construct that encapsulates a collection of procedures and variables. The key feature of a monitor is that the execution of the procedures inside the monitor is mutually exclusive. No procedure inside a monitor can begin execution while another procedure is executing.

Although they were not part of Ada 83, Ada 95 added the syntax of protected objects that provide monitor-style synchronization. Now we are ready to return to the ATM example that we introduced earlier. By replacing the package that defined a singleton object used previously with a protected object, we ensure that the procedures Deposit and Withdraw execute in a mutually exclusive way, and no race condition can occur. Consider this modified code. Only the necessary changes are highlighted.

```
procedure No_Race_Condition is
    protected Account is
        procedure Deposit(Amount: in Integer);
        procedure Withdraw(Amount: in Integer; Sufficient_Funds
            Boolean);
    private
        Balance: Integer := 0;
    end Account;
    protected body Account is separate;
    task type ATM;
    task body ATM is separate;
    -- Task body is the same as before
    ATM_1, ATM_2: ATM;
begin
    null;
end No_Race_Condition;

separate (No_Race_Condition)
protected body Account is
    procedure Deposit(Amount: in Integer) is
    begin
        Balance := Balance + Amount;
    end Deposit;
    procedure Withdraw(Amount: in Integer; Sufficient_Funds: ou
        Boolean) is
    begin
        if Amount <= Balance then
            Balance := Balance - Amount;
            Sufficient_Funds := True;
        end if;
        Sufficient_Funds := False;
```

```
    end Withdraw;
end Account;
```

One noteworthy comment is that the use of the reserved word protected in this context is completely unrelated to the way it is used in C++, Java, or C#.

Ada protected objects provide a second level of synchronization that goes beyond what is required of a monitor. This second level accommodates cooperation synchronization and requires the use of a syntax that uses entries rather than procedures and entry barriers to control the synchronization.

To illustrate these two levels of synchronization, we have chosen the classical synchronization problem—a bounded buffer. The bounded buffer is a first-in first-out queue implemented with a fixed-size array. The operations are enqueue and dequeue. There are two tasks that access this buffer—a producer task that reads in characters and enqueues them and a consumer task that dequeues characters and outputs them.

Using a protected object to encapsulate the bounded buffer ensures that the execution of enqueue and dequeue are mutually exclusive. For this problem, however, a second level of synchronization is needed. We must ensure that the producer becomes blocked whenever it attempts to enqueue a character and the queue is full. We must also ensure that the consumer task becomes blocked whenever it attempts to dequeue a character and the buffer is empty. Let's examine the code that solves this problem.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
with Text_IO;
procedure Producer_Consumer_1 is
    Size: constant Integer := 100;
    type Indexes is mod Size;
    type Buffers is array(Indexes) of Character;
    protected Bounded_Buffer is
        entry Enqueue (Char: in Character);
        entry Dequeue (Char: out Character);
    private
        Front, Back: Indexes := 0;
        Filled: Integer range 0..Size := 0;
        Buffer: Buffers;
    end Bounded_Buffer;
    protected body Bounded_Buffer is
        entry Enqueue (Char: in Character) when Filled < Size i
        begin
            Buffer(Back) := Char;
            Back := Back + 1;
            Filled := Filled + 1;
        end Enqueue;
        entry Dequeue (Char: out Character) when Filled > 0 is
        begin
            Char := Buffer(Front);
            Front := Front + 1;
            Filled := Filled - 1;
        end Dequeue;
    end Bounded_Buffer;
    task Producer;
    task body Producer is
        Input_Char: Character;
    begin
        loop
            Text_IO.Get(Input_Char);
```

```
            Bounded_Buffer.Enqueue(Input_Char);
        end loop;
    end Producer;
    task Consumer;
    task body Consumer is
        Output_Char: Character;
    begin
        loop
            Bounded_Buffer.Dequeue(Output_Char);
            Text_IO.Put(Output_Char);
        end loop;
    end Consumer;
begin
    null;
end Producer_Consumer_1;
```

Message-Passing

The second synchronization technique we explore is message passing. This technique can be implemented in various ways. The first variation is whether the communication is synchronous or asynchronous. When we call a friend on the telephone, we are engaging in synchronous communication. We must wait for our friend to answer, before we can have a conversation. When we send a letter using the postal service, we are engaging in asynchronous communication. In addition, electronic mail and leaving messages on telephone-answering machines constitute asynchronous communication.

The second major variation is whether the communication uses symmetric protocol or asymmetric protocol. These protocols are also a part of our everyday experience. Most telephone communication has an asymmetric protocol. The caller selects whom he calls, but the receiver takes calls from anyone. If, however, we have a dedicated line between two parties, when the phone rings, only one person can be calling. This is a symmetric communication protocol.

Now that we have explored the major variations within the message-passing model, we are prepared to discuss the techniques that Ada uses. Ada 83 introduced the first message-passing technique, called *rendezvous*, which is synchronous

communication with an asymmetric protocol. In Ada 95, an asynchronous form was added. We plan to focus on the former technique.

To illustrate rendezvous message-passing, we return to the producer-consumer problem that requires two levels of synchronization and implement it using a task in place of the protected object that we used previously with rendezvous for synchronization.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
with Text_IO;
procedure Producer_Consumer_2 is
    Size: constant Integer := 100;
    type Indexes is mod Size;
    type Buffers is array(Indexes) of Character;
    task Bounded_Buffer is
        entry Enqueue (Char: in Character);
        entry Dequeue (Char: out Character);
    end Buffer;
    task body Bounded_Buffer is
        Front, Back: Indexes := 0;
        Filled: Integer range 0..Size := 0;
        Buffer: Buffers;
    begin
        loop
            select
                when Filled < Size =>
                accept Enqueue (Char: in Character) do
                    Buffer(Back) := Char;
                    Back := Back + 1;
                    Filled := Filled + 1;
                end Enqueue;
            or
                when Filled > 0 =>
                accept Dequeue (Char: out Character) do
                    Char := Buffer(Front);
                    Front := Front + 1;
                    Filled := Filled - 1;
                end Dequeue;
            end select;
        end loop;
    end Bounded_Buffer;
    task Producer;
```

```
    task body Producer is
        Input_Char: Character;
    begin
        loop
            Text_IO.Get(Input_Char);
            Bounded_Buffer.Enqueue(Input_Char);
        end loop;
    end Producer;
    task Consumer;
    task Consumer is
        Output_Char;
    begin
        loop
            Bounded_Buffer.Dequeue(Output_Char);
            Text_IO.Put(Output_Char);
        end loop;
    end Consumer;
begin
    null;
end Producer_Consumer_2;
```

Using rendezvous synchronization, the guards on the accept statements control the cooperative synchronization. The mutual exclusion is achieved by the fact that a task has its own thread of control and can be doing only one thing at a time, so when it is accepting the request of one task, it will not be ready to accept any other requests.

Semaphores

We have saved a discussion of semaphores for last because they are the most primitive synchronization technique and are often characterized as the *goto* form of synchronization control. We highlight the fact that semaphores are a very low-level technique because the potential for misuse is great, and it is easy to create problems such as deadlock. It is for that reason the Ada designers chose not to

implement such a synchronization technique. We will demonstrate that although semaphores are not built into Ada, they can be implemented using both of Ada's synchronization techniques.
There are two types of semaphores—binary semaphores and counting semaphores. We confine our discussion to the former. A binary semaphore can be represented as a Boolean value. Do not be misled, however, into thinking that a binary semaphore is only a Boolean variable.

Its state can be changed only by performing one of two operations—secure or release. The secure operation checks the Boolean value. If it is false, it sets it to true, and the secure operation is complete. If it is already true, however, the task initiating the call is blocked, and the task is queued to await the release. A call to release causes the value to be set to true and causes one of the tasks queued, awaiting release, to be returned to the ready state in order to complete its secure operation. It should be clear that a semaphore is a great deal more than just a Boolean value. Its real essence is defined by its two operations.

First, let's consider how a binary semaphore can be implemented using a protected object.

Move your mouse over the highlighted parts of the protected object to see the explanations of those parts.

```
protected Binary_Semaphore is
    entry Secure;
    procedure Release;
private
    Available: Boolean := True;
end Binary_Semaphore;
protected body Binary_Semaphore is
    entry Secure when Available is
    begin
        Available := False;
    end Secure;
    procedure Release is
    begin
        Available := True;
    end Release;
end Binary_Semaphore;
```

Next, let's examine how a binary semaphore can be constructed using a task and message-passing synchronization.

Move your mouse over the highlighted part of the task to see the explanation of that part.

```
task Binary_Semaphore is
    entry Secure;
    entry Release;
end Binary_Semaphore;

task body Binary_Semaphore is
begin
    loop
        accept Secure;
        accept Release;
    end loop;
end Binary_Semaphore;
```

Figure 4.3 is a finite-state diagram that illustrates the behavior of this task. Notice that no flag is needed because the state of the object captures when the semaphore is or is not available.

Figure 4.3
Finite-State Diagram for a Binary Semaphore Task

IV. Exceptions

Exceptions are erroneous conditions that are unexpected during the normal execution of a program and that require special handling when they do occur. It is particularly important that languages that are designed to handle critical applications provide this capability. Ada, which was developed with embedded system development in mind, was one of the first languages to include it.

In our discussion of exceptions, we will consider the way exceptions are declared, the way they are thrown, the way they are caught, and whether the language provides syntax for subprograms to designate exceptions that they might throw. In

our discussion, we plan to compare three languages, Ada, C++, and Java.

A. Declaring Exceptions

Although only Ada and Java have some predefined exceptions, all three languages allow the user to define additional ones. What we consider first is the way these user-defined exceptions are declared.

In Ada, exception is a reserved word, and we declare exceptions by including this reserved word, as illustrated below:

```
MyException: exception;
```

In C++, there is no distinction between exceptions and any other kind of object, so in C++, any object can be treated as an exception object.

In Java, Exception is a predefined class, and all user-defined exception classes must be defined as classes derived from this class. The parent class of Exception is Throwable, which defines the broadest class whose objects can be thrown. In Java, an exception class declaration might look as follows:

```
class MyException extends Exception
{
}
```

Although exception classes can have instance methods and methods like any other class, they often have neither.

B. Throwing Exceptions

When an exceptional condition is detected, an exception is thrown. In Ada, a raise statement is used to accomplish this objective. Such a statement might look as follows:

```
raise MyException;
```

In both C++ and Java, a throw statement is used. In both languages, exception names are classes, so an instance of the class must be thrown. These instances are typically anonymous. In Java, it might look as follows:

```
throw new MyException();
```

Because C++ permits stack-dynamic objects, the exception instance is often created with a call to the constructor as follows:

```
throw MyException();
```

C. Catching Exceptions

When an exception has been thrown, ideally it will be caught within the program. In Ada, exception handlers can be associated with any begin-end pair. Such pairs might delimit blocks, subprograms, or packages, among others. A block that has such a handler might look as follows:

```
begin
    -- code that might raise an exception
when exception_name =>
    --code to handle the exception
end;
```

Any number of when clauses may be provided to catch various exceptions.

Both C++ and Java require that a try statement be used to catch exceptions. Its code looks as follows:

```
try
{
    // code that might throw an exception
}
catch (ExceptionClass exceptionObject)
{
    // code to catch exception
}
```

As with Ada, any number of catch blocks may be provided. One difference, however, results from the fact that in C++ and Java, exceptions are objects. A catch block will catch any exception of the specified exception class or any class derived from it.

In each of these languages, exceptions are propagated if they are not caught. When nested blocks or nested try statements are used, the exceptions propagate progressively to the next outermost block or statement. Once they reach the subprogram level they are propagated according to the dynamic chain of subprogram calls. If an exception is never caught, the program terminates.

One final difference among languages is that the Java try statement allows an optional finally block. When present, the code inside this block is executed last and is executed regardless of whether or not an exception is thrown.

D. Designating Thrown Exceptions

One final issue is whether the language allows the exceptions that might be thrown in a method to be included after its signature. Ada does not allow such a designation, but both C++ and Java do. There are two differences, however. First, in C++, the reserved word throw is reused for this purpose, whereas Java included a new reserved word throws. In Java, such a designation might look like the following example in which we highlighted the throws clause:

```
void push(Object item) throws StackFull
```

The second difference is that Java distinguishes between unchecked exceptions and checked exceptions. If a checked exception might be thrown in a method and that method does not catch it, it must be designated in the throws clause of that method.

Return to top of page